Stop Collecting Everything

Komplette Podman-Anleitung: Astronomy Shop, Jaeger, Sampling und AI/MCP Sampling Advisor

ZIEL Trace-Volumen mit OpenTelemetry messen und reduzieren, wertvolle Traces schützen, KI-Empfehlungen über einen read-only MCP-Server erzeugen und Before/After nachweisen.

Baustein	Technik
Runtime	Podman Desktop und Podman Machine
Demo	OpenTelemetry Astronomy Shop
Analyse	Jaeger, Prometheus, Span Metrics
KI	MCP Sampling Advisor, read-only
Optimierung	Probabilistic und Tail Sampling
Plattform	macOS, empfohlen 16 GB RAM

Stand: Oktober 2026. Lokales Lern- und Event-Lab, keine Produktionsdimensionierung.

1. Zielarchitektur

Erfolgreiche Anwendungen erzeugen mit wachsendem Traffic, Services und Abhängigkeiten mehr Telemetrie. Die Optimierung reduziert Low-value-Traffic möglichst früh und erhält Fehler, langsame Requests und geschäftskritische Flows.

```
Browser -> Astronomy Shop -> Microservices -> OTLP
  -> OpenTelemetry Collector
       -> Processing / Redaction
       -> Span Metrics -> Prometheus -> MCP -> AI
       -> Sampling -> Jaeger
  -> Human Review -> neue Sampling Policy
```

LOOP Observe → Measure → Analyze → Recommend → Review → Optimize → Measure again

2. Podman vorbereiten

Podman Desktop bevorzugt über den offiziellen macOS-DMG-Installer installieren. Auf macOS laufen Linux-Container in einer Podman Machine.

Wichtig: Wenn podman-machine-default bereits existiert, podman machine init nicht erneut ausführen. Stattdessen podman machine list, podman machine inspect podman-machine-default und podman machine start verwenden.

```
podman --version
podman machine list
# Falls noch keine Machine existiert:
podman machine init --cpus 4 --memory 6144 --disk-size 60
podman machine start
podman info
podman compose version
```

COMPOSE Podman Desktop → Settings → Resources → Compose → Setup öffnen. Anschließend im Terminal mit podman compose version prüfen. podman compose verwendet einen externen Compose-Provider.

- Git
- Python 3.10+ nur für lokale MCP-Ausführung; der vorgesehene MCP-Container verwendet Python 3.12
- Node.js/npx nur für MCP Inspector
- Mindestens 14 GB freier Speicher, besser mehr für Images

```
git --version
python3 --version
node --version
npx --version
node --version
npx --version
lsof -i :8080
lsof -i :8000
```

3. Astronomy Shop starten

```
mkdir -p ~/Documents/otel-cost-workshop
cd ~/Documents/otel-cost-workshop
git clone https://github.com/open-telemetry/opentelemetry-demo.git
cd opentelemetry-demo
git switch -c podman-ai-sampling-lab

podman compose --project-name otel-demo \
 --env-file .env --env-file .env.override \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 up --force-recreate --remove-orphans --detach

podman compose --project-name otel-demo \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml ps
podman ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
podman stats
```

WICHTIG Nicht nur compose.yaml starten. Für Jaeger, Prometheus und weitere Backends müssen die Observability- und Extras-Dateien mitgegeben werden.

3.1 Podman-Override für Grafana

Die getestete Podman/macOS-Umgebung benötigte für Grafana mehr als das in compose.observability.yaml gesetzte Limit von 175M. Erstelle deshalb die Datei compose.podman.yaml mit folgendem Inhalt:

```
services:
 grafana:
  deploy:
   resources:
    limits:
     memory: 512M
```

Nimm compose.podman.yaml in jeden Compose-Start-, Status-, Logs- und Stop-Befehl auf: -f compose.podman.yaml

```
podman compose --project-name otel-demo --env-file .env --env-file .env.override -f compose.yaml -f
compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml up --force-recreate --remove-orphans --detach
```

Nach dem Start Grafana intern prüfen:

```
podman exec grafana curl http://127.0.0.1:3000/api/health
# Erwartung: "database": "ok"
# Browser: http://localhost:8080/grafana/
```

4. UIs und erste Traces

UI	Adresse
Shop	http://localhost:8080/
Jaeger	http://localhost:8080/jaeger/ui/
Grafana	http://localhost:8080/grafana/ Über den Frontend-Proxy öffnen. Den dynamischen Port aus podman port grafana nicht als Browser-URL verwenden.
Prometheus	http://localhost:9090/
Load Generator	http://localhost:8080/loadgen/
Feature Flags	http://localhost:8080/feature

1. Produkt öffnen, Warenkorb und Checkout durchführen.
2. Journey fünf- bis zehnmal wiederholen.
3. In Jaeger frontend, checkout oder product-catalog wählen.
4. Trace öffnen und Services, Dauer, Attribute und Fehler prüfen.
5. Feature Flag für ein kontrolliertes Error- oder Slow-Szenario aktivieren und danach zurücksetzen.

TESTREGEL Zwischen Varianten nur Sampling ändern. Last, Dauer, User Journey und Suchzeitraum bleiben gleich.

5. Baseline

```
cp src/otel-collector/otelcol-config.yml \
 src/otel-collector/otelcol-config.original.yml
date
podman stats --no-stream
podman compose --project-name otel-demo \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 logs --tail 100 otel-collector > baseline-collector.log
```

Messgröße	Baseline	50 %	10 %	Tail	AI Policy
Testdauer					
Calls/Spans					
Bytes, falls messbar					
Errors sichtbar					
Slow Traces sichtbar					
Collector CPU/RAM					

Jaeger-Treffer sind eine Demo-Messgröße, nicht automatisch eine Abrechnungsmetrik. Für Ingest exportierte Spans, Calls oder Bytes bevorzugen.

6. Probabilistisches Sampling

```
processors:
  probabilistic_sampler:
    sampling_percentage: 50

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [resource_detection, memory_limiter,
        transform/sanitize_spans, gen_ai_normalizer,
        transform/redact_sensitive_data, redaction,
        probabilistic_sampler]
      exporters: [debug, span_metrics]
```

Vorhandene Prozessoren nicht ersetzen. Sampler ergänzen, Stack mit demselben Podman-Compose-Befehl neu erstellen und Logs prüfen:

```
podman compose --project-name otel-demo \
 --env-file .env --env-file .env.override \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 up --force-recreate --remove-orphans --detach
podman compose --project-name otel-demo \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 logs otel-collector | grep -iE "error|invalid|failed|unknown"
```

Test mit 50 %, danach mit sampling_percentage: 10 wiederholen.

7. Tail Sampling

```
processors:
  tail_sampling:
    decision_wait: 10s
    num_traces: 5000
    expected_new_traces_per_sec: 100
    policies:
      - name: keep-errors
        type: status_code
        status_code: {status_codes: [ERROR]}
      - name: keep-slow
        type: latency
        latency: {threshold_ms: 1000}
      - name: sample-normal
        type: probabilistic
        probabilistic: {sampling_percentage: 10}
```

Probabilistic Sampler aus Definition und Pipeline entfernen, tail_sampling einsetzen, Stack neu erstellen. Normalen, fehlerhaften und langsamen Traffic testen und mindestens decision_wait abwarten.

ARCHITEKTUR Alle Spans eines Trace müssen dieselbe Collector-Instanz erreichen. Tail Sampling puffert Traces und benötigt Speicher.

8. Span Metrics und PromQL

Zuerst tatsächliche Metrik- und Labelnamen entdecken, weil die Prometheus-Übersetzung versionsabhängig sein kann.

```
{__name__=~".*span.*calls.*"}
{__name__=~".*span.*duration.*"}
curl -s 'http://localhost:9090/api/v1/label/__name__/values' \
 | python3 -m json.tool
```

Beispiele nach Bestätigung der Namen:

```
sum by (service_name, span_name)(rate(traces_span_metrics_calls_total[5m]))
100 * sum by (service_name,
span_name)(rate(traces_span_metrics_calls_total{status_code="STATUS_CODE_ERROR"}[5m]))
/ sum by (service_name, span_name)(rate(traces_span_metrics_calls_total[5m]))
histogram_quantile(0.95,sum by
(le,service_name,span_name)(rate(traces_span_metrics_duration_milliseconds_bucket[5m])))
```

KARDINALITÄT Nur stabile, normalisierte Labels verwenden. Keine IDs oder dynamischen URL-Pfade.

9. MCP Sampling Advisor bauen

```
mkdir -p mcp-sampling-advisor
cd mcp-sampling-advisor
# requirements.txt
mcp[cli]>=2.2,<3
httpx>=0.27,<1

# Containerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY server.py .
EXPOSE 8000
CMD ["python", "server.py"]
```

9.1 server.py

```
import os, httpx
from mcp.server import MCPServer
mcp=MCPServer("OTel Sampling Advisor")
PROM=os.getenv("PROM_URL","http://prometheus:9090")
CALLS=os.getenv("CALLS_METRIC","traces_span_metrics_calls_total")
DUR=os.getenv("DURATION_BUCKET_METRIC","traces_span_metrics_duration_milliseconds_bucket")
SVC=os.getenv("SERVICE_LABEL","service_name")
SPAN=os.getenv("SPAN_LABEL","span_name")
STATUS=os.getenv("STATUS_LABEL","status_code")
ERR=os.getenv("ERROR_VALUE","STATUS_CODE_ERROR")
async def q(expr):
 async with httpx.AsyncClient(timeout=20) as c:
  r=await c.get(f"{PROM}/api/v1/query",params={"query":expr});r.raise_for_status()
  return r.json()["data"]["result"]
def out(rows): return [{"labels":r.get("metric",{}),"value":float(r["value"][1])} for r in rows]
@mcp.tool()
async def top_operations(window:str="5m",limit:int=15):
 return out(await q(f"topk({limit},sum by ({SVC},{SPAN})(rate({CALLS}[{window}])))"))
@mcp.tool()
async def error_hotspots(window:str="5m",limit:int=15):
 e=f'100*sum by ({SVC},{SPAN})(rate({CALLS}{{{STATUS}="{ERR}"}}[{window}]))/clamp_min(sum by
({SVC},{SPAN})(rate({CALLS}[{window}])),0.000001)'
 return out(await q(f"topk({limit},{e})"))
@mcp.tool()
async def slow_operations(window:str="5m",limit:int=15):
 e=f"histogram_quantile(0.95,sum by (le,{SVC},{SPAN})(rate({DUR}[{window}])))"
 return out(await q(f"topk({limit},{e})"))
@mcp.tool()
async def sampling_candidates(window:str="5m",limit:int=20):
 return {"volume":await top_operations(window,limit),"errors":await error_hotspots(window,limit),"slow":await
slow_operations(window,limit),"rules":["keep errors","keep slow","protect critical","human review"]}
if __name__=="__main__": mcp.run("streamable-http",host="0.0.0.0",port=8000)
```

9.2 Compose-Erweiterung für MCP

```
# compose.mcp.yaml
services:
  mcp-sampling-advisor:
    build:
      context: ./mcp-sampling-advisor
      dockerfile: Containerfile
    environment:
      PROM_URL: http://prometheus:9090
      CALLS_METRIC: ${CALLS_METRIC:-traces_span_metrics_calls_total}
      DURATION_BUCKET_METRIC: ${DURATION_BUCKET_METRIC:-traces_span_metrics_duration_milliseconds_bucket}
      SERVICE_LABEL: ${SERVICE_LABEL:-service_name}
      SPAN_LABEL: ${SPAN_LABEL:-span_name}
      STATUS_LABEL: ${STATUS_LABEL:-status_code}
      ERROR_VALUE: ${ERROR_VALUE:-STATUS_CODE_ERROR}
    ports: ["8000:8000"]
    depends_on: [prometheus]
```

Echte Namen aus Prometheus in .env.override setzen. Danach:

```
podman compose --project-name otel-demo \
 --env-file .env --env-file .env.override \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 -f compose.mcp.yaml up --build --force-recreate --remove-orphans --detach
podman compose --project-name otel-demo \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 -f compose.mcp.yaml logs --tail 100 mcp-sampling-advisor
```

10. MCP und KI testen

```
npx -y @modelcontextprotocol/inspector
# Mit http://localhost:8000/mcp verbinden
```

- top_operations
- error_hotspots
- slow_operations
- sampling_candidates

Systemanweisung:

```
Du bist ein Observability Sampling Advisor.
Nutze nur MCP-Daten und erfinde keine Werte.
Erhalte Errors, langsame Traces und geschäftskritische Flows.
Empfehle aggressives Sampling nur bei hohem Volumen,
niedriger Fehlerquote und niedriger Latenz.
Liefere Evidenz, Risiko, Policy-Idee, Validierung und Rollback.
Führe keine Änderung aus; ein Mensch entscheidet.
```

11. Endpoint-spezifische Policy

```
tail_sampling:
  policies:
    - name: keep-errors
      type: status_code
      status_code: {status_codes: [ERROR]}
    - name: keep-slow
      type: latency
      latency: {threshold_ms: 1000}
    - name: keep-critical-routes
      type: string_attribute
      string_attribute:
        key: http.route
        values: [/api/checkout, /api/payment]
    - name: sample-normal
      type: probabilistic
      probabilistic: {sampling_percentage: 10}
```

WICHTIG Routen sind Platzhalter. Nur tatsächlich vorhandene, normalisierte Attribute verwenden. Vollständigen Drop erst nach fachlichem Review.

12. Before/After

KPI	Before	After	Ziel
Calls/Spans			reduziert
Bytes			reduziert
Errors sichtbar			ja
Slow Traces sichtbar			ja
Kritische Flows			ja
MTTD/MTTR-Test			nicht verschlechtert

```
Reduktion % = (Baseline - After) / Baseline * 100
Variable Einsparung = vermiedene Ingest-Einheiten * Vertragspreis
```

Ohne Vertragsdaten nur technische Reduktion ausweisen. Fixkosten, Mindestabnahmen, Retention, Egress und Rabatte separat betrachten.

13. Troubleshooting und Cleanup

Problem	Prüfung
Compose fehlt	Podman Desktop Compose-Erweiterung und podman compose version
Machine gestoppt	podman machine start
Keine Traces	Traffic, Zeitraum, decision_wait, Collector-Logs
MCP leer	PromQL, Namen, Flush-Intervall, MCP-Logs
Volume-Rechte	Bind Mount prüfen; bevorzugt benannte Volumes nutzen
Port belegt	lsof und alternative Host-Ports
Grafana startet nicht / Port 3000 verweigert	Grafana-Limit per compose.podman.yaml auf 512M erhöhen; Stack neu erstellen; podman exec grafana curl http://127.0.0.1:3000/api/health prüfen.
Grafana leitet auf localhost/grafana um	Grafana nicht über den dynamischen Host-Port öffnen. Browser-URL: http://localhost:8080/grafana/ über den Frontend-Proxy.
Prometheus über Port 8080 nicht erreichbar	Prometheus direkt über http://localhost:9090 öffnen und API-Aufrufe gegen Port 9090 ausführen.

```
# Rollback
cp src/otel-collector/otelcol-config.original.yml src/otel-collector/otelcol-config.yml

# Stop mit MCP
podman compose --project-name otel-demo \
 --env-file .env --env-file .env.override \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 -f compose.mcp.yaml down --remove-orphans

# Nur wenn Daten gelöscht werden dürfen: down --volumes
podman system df
podman machine stop
```

14. Event-Checkliste und Responsible AI

- Podman Desktop und Machine laufen.
- podman compose version funktioniert.
- Images sind vorab gepullt.
- Shop, Jaeger, Prometheus und Feature Flags sind erreichbar.
- Baseline und Error-/Slow-Szenario wurden getestet.
- MCP ist read-only und liefert plausible Daten.
- Human Review und Rollback sind vorbereitet.
- Für Produktion: TLS, Auth, Netzwerk-Policies, Audit Logging und Rate Limits ergänzen.
- Tail Sampling bei Skalierung trace-affin verteilen und Speicher beobachten.

15. Referenzen

Grafana Reverse Proxy und Subpath: https://grafana.com/tutorials/run-grafana-behind-a-proxy/

Grafana Health API: https://grafana.com/docs/grafana/latest/developer-resources/api-reference/http-api/api-legacy/other/

Podman Installation: https://podman.io/docs/installation

Podman Compose: https://docs.podman.io/en/latest/markdown/podman-compose.1.html

Podman Desktop macOS: https://podman-desktop.io/docs/installation/macos-install

Podman Docker-Kompatibilität: https://podman-desktop.io/docs/migrating-from-docker/managing-docker-compatibility

OpenTelemetry Demo: https://github.com/open-telemetry/opentelemetry-demo

OpenTelemetry Docker Deployment als Upstream-Compose-Referenz: https://opentelemetry.io/docs/demo/docker-deployment/

Tail Sampling: https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/processor/tailsamplingprocessor/README.md

Span Metrics: https://github.com/open-telemetry/opentelemetry-collector-contrib/blob/main/connector/spanmetricsconnector/README.md

MCP Python SDK: https://py.sdk.modelcontextprotocol.io/

16. Schritt-für-Schritt-Checkliste für macOS

Diese Checkliste ist für Teilnehmer gedacht, die das Lab auf einem Mac mit Podman Desktop ausführen. Die Schritte führen von einem vorbereiteten Mac bis zum vollständigen Cleanup.

A. Mac vorbereiten

- ☐ **CPU-Architektur prüfen.**

Apple-Menü → Über diesen Mac. Für das Event sollte jeder wissen, ob Apple Silicon oder Intel verwendet wird, damit passende Downloads gewählt werden können.

- ☐ **Mindestens 14 GB freien Speicher bereitstellen.**

Für Images und Lab-Dateien empfiehlt sich zusätzliche Reserve.

```
df -h ~

uname -m
```

- ☐ **Ports 8080 und 8000 prüfen.**

```
lsof -i :8080

lsof -i :8000
```

B. Podman Desktop installieren

- ☐ **Podman Desktop als .dmg von der offiziellen Podman-Desktop-Seite laden.**

Der .dmg-Installer ist laut offizieller Dokumentation der empfohlene Installationsweg auf macOS.

- ☐ **Podman Desktop nach Applications ziehen und starten.**
- ☐ **Onboarding starten und Podman installieren.**
- ☐ **Podman Machine im Onboarding erstellen.**
- ☐ **Compose CLI im Onboarding installieren.**

Alternativ später: Settings → Resources → Compose → Setup.

- ☐ **Installation im Terminal prüfen.**

```
podman --version

podman machine list

podman info

podman compose version
```

C. Podman Machine für das Lab dimensionieren

- ☐ **Vorhandene Machine prüfen.**

```
podman machine list
```

```
podman machine inspect
```

- ☐ **Falls noch keine Machine existiert, Lab-Machine anlegen.**

Die folgenden Werte sind eine Empfehlung für dieses Lab, keine offizielle Produktionsdimensionierung.

```
podman machine init --cpus 4 --memory 6144 --disk-size 60

podman machine start
```

- ☐ **Wenn bereits eine Machine existiert, nicht blind eine zweite Default-Machine erstellen.**
- ☐ **Wenn die Meldung „podman-machine-default already exists“ erscheint, keine neue Machine initialisieren. Bestehende Machine mit podman machine start verwenden.**

Entweder die vorhandene Machine verwenden oder bewusst einen Namen für eine separate Lab-Machine vergeben.

- ☐ **Podman-Verbindung validieren.**

```
podman info

podman run --rm quay.io/podman/hello
```

D. Werkzeuge prüfen

- ☐ **Git verfügbar.**
- ☐ **Python verfügbar, wenn der MCP Advisor außerhalb eines Containers getestet werden soll.**
- ☐ **Node.js und npx verfügbar, wenn MCP Inspector verwendet wird.**

```
git --version

python3 --version
node --version
npx --version
node --version
npx --version

node --version

npx --version
```

E. OpenTelemetry Demo herunterladen

- ☐ **Arbeitsverzeichnis anlegen und Repository klonen.**

```
mkdir -p ~/Documents/otel-cost-workshop

cd ~/Documents/otel-cost-workshop

git clone https://github.com/open-telemetry/opentelemetry-demo.git

cd opentelemetry-demo

git switch -c podman-ai-sampling-lab
```

- ☐ **Kerndateien prüfen.**

```
ls -la compose.yaml compose.observability.yaml compose.extras.yaml .env
```

F. Astronomy Shop mit Podman starten

- ☐ **Podman Machine läuft.**

```
podman machine start

podman machine list
```

- ☐ **Demo mit Observability-Komponenten starten.**

```
podman compose --project-name otel-demo \

 --env-file .env --env-file .env.override \

 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \

 up --force-recreate --remove-orphans --detach
```

- ☐ **Containerstatus prüfen.**

```
podman compose --project-name otel-demo \

 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml ps

podman ps
```

- ☐ **Ressourcen beobachten.**

```
podman stats
```

G. Alle Oberflächen prüfen

- ☐ **Shop: http://localhost:8080/**
- ☐ **Jaeger: http://localhost:8080/jaeger/ui/**
- ☐ Grafana: http://localhost:8080/grafana/ über den Frontend-Proxy über den Frontend-Proxy
- ☐ **Prometheus: http://localhost:9090/**
- ☐ **Load Generator: http://localhost:8080/loadgen/**
- ☐ **Feature Flags: http://localhost:8080/feature**

H. Baseline-Test

- ☐ **Shop-Journey mehrfach durchführen: Produkt → Warenkorb → Checkout.**
- ☐ **Trace in Jaeger öffnen und Service-Kette prüfen.**
- ☐ **Collector-Konfiguration sichern.**

```
cp src/otel-collector/otelcol-config.yml \

 src/otel-collector/otelcol-config.original.yml
```

- ☐ **Baseline-Werte in der Mess-Tabelle eintragen.**

I. Sampling-Übungen

- ☐ **Probabilistic Sampling mit 50 % konfigurieren und Stack neu erstellen.**
- ☐ **Identischen Test durchführen und Werte erfassen.**
- ☐ **Sampling auf 10 % ändern und identischen Test wiederholen.**
- ☐ **Probabilistic Sampler entfernen und Tail Sampling konfigurieren.**
- ☐ **Normalen Traffic, Error-Szenario und Slow-Szenario testen.**
- ☐ **Prüfen, dass Fehler und langsame Traces weiterhin sichtbar sind.**

J. MCP Sampling Advisor

- ☐ **Prometheus-Metrik- und Labelnamen zuerst prüfen.**
- ☐ **mcp-sampling-advisor-Verzeichnis, requirements.txt, Containerfile und server.py gemäß Kapitel 9 anlegen.**
- ☐ **compose.mcp.yaml gemäß Kapitel 9 anlegen.**
- ☐ **MCP-Komponente starten.**

```
podman compose --project-name otel-demo \

 --env-file .env --env-file .env.override \

 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \

 -f compose.mcp.yaml up --build --force-recreate --remove-orphans --detach
```

- ☐ **MCP-Logs prüfen.**

```
podman compose --project-name otel-demo \

 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \

 -f compose.mcp.yaml logs --tail 100 mcp-sampling-advisor
```

- ☐ **Optional MCP Inspector starten und http://localhost:8000/mcp verbinden.**

```
npx -y @modelcontextprotocol/inspector
```

- ☐ **top_operations, error_hotspots, slow_operations und sampling_candidates testen.**

K. KI-Optimierung durchführen

- ☐ **High-volume-Operationen bestimmen.**
- ☐ **Fehlerrate und p95-Latenz gegenüberstellen.**
- ☐ **Geschäftskritische Flows manuell markieren.**
- ☐ **KI-Vorschlag in KEEP, KEEP SLOW, KEEP MORE, SAMPLE und gegebenenfalls DROP einordnen.**
- ☐ **Policy durch einen Menschen reviewen.**
- ☐ **Collector-Konfiguration ändern und Stack neu erstellen.**
- ☐ **Identisches Traffic-Szenario erneut ausführen.**
- ☐ **Before/After dokumentieren und MTTD-/MTTR-relevante Sichtbarkeit prüfen.**

L. Reset und Cleanup

- ☐ **Collector-Konfiguration bei Bedarf zurücksetzen.**

```
cp src/otel-collector/otelcol-config.original.yml src/otel-collector/otelcol-config.yml
```

- ☐ **Stack beenden.**

```
podman compose --project-name otel-demo \
 --env-file .env --env-file .env.override \
 -f compose.yaml -f compose.observability.yaml -f compose.extras.yaml -f compose.podman.yaml \
 -f compose.mcp.yaml down --remove-orphans
```

- ☐ **Nur wenn Lab-Daten vollständig gelöscht werden dürfen: zusätzlich --volumes verwenden.**
- ☐ **Speichernutzung prüfen und Podman Machine beenden.**

```
podman system df

podman machine stop
```

M. Go/No-Go vor dem Event

- ☐ **podman compose version funktioniert**
- ☐ **Podman Machine startet sauber**
- ☐ **Alle Images sind lokal verfügbar**
- ☐ **Shop und Jaeger öffnen sich**
- ☐ **Prometheus liefert Span Metrics**
- ☐ **50-%- und 10-%-Sampling getestet**
- ☐ **Tail Sampling behält Error- und Slow-Traces**
- ☐ **MCP Tools liefern plausible Werte**
- ☐ **Rollback wurde getestet**
- ☐ **Backup-Demo und Screenshots sind verfügbar**

macOS-Quellen: Podman Desktop Installation, Podman Compose Setup und Podman Installation. Siehe Referenzen in Kapitel 15.